



ESCOM

Ingeniería en Inteligencia Artificial

Algoritmos y Estructuras de Datos

Grupo 2BM2

Práctica 2

Ordenamiento por inserción, burbuja, selección y mezcla

Integrantes

Josué Agudelo Mancera

Jesús Gabriel Ramírez González

Profesora

Cecilia Albortante Morato

Introducción

En esta práctica implementamos en C los ordenamientos por inserción, burbuja, selección y mezcla. Cada método reorganiza un arreglo de enteros de menor a mayor, pero utiliza una estrategia distinta. Mediante el menú podemos elegir el algoritmo y observar cómo cambia el arreglo hasta obtener el resultado.

La actividad indicada por la profesora solicita definir el tamaño de entrada, generar enteros aleatorios entre -100 y 100, mostrar los datos antes y después del ordenamiento y colocar cada algoritmo en una función. Para comprobar el comportamiento también utilizamos las opciones de arreglo ordenado e invertido que ofrece el programa.

Desarrollo del programa

Programa1.c permite inicializar el arreglo, consultar su contenido original, elegir uno de los cuatro métodos y salir. La cantidad aceptada va de 1 a 1 000 000. Los valores aleatorios se generan con `rand() % 201 - 100`; la entrada ordenada contiene los valores de 1 a n y la invertida los presenta de n a 1.

```
MENU

1). Inicializar arreglo.
2). Ver arreglo generado.
3). Ordenar el arreglo generado.
4). Salir del programa.

Elige el algoritmo de ordenamiento:
1) Insercion    2) Burbuja    3) Seleccion    4) Mezcla
```

Figura 1. Fragmentos de los menús del programa. La elección del algoritmo se realiza después de inicializar el arreglo.

Antes de cada ordenamiento se copia el arreglo original en el arreglo de trabajo. Esto permite aplicar varios métodos a los mismos datos. Con hasta diez elementos se muestran los pasos; con más de diez se ejecuta la versión sin pasos y se mide el tiempo. También se calcula la mediana del resultado.

Casos utilizados

Las salidas presentadas incluyen una entrada aleatoria con negativos y repetidos, otra invertida, una ya ordenada y un arreglo de un elemento. En cada prueba verificamos que el resultado estuviera en orden ascendente, conservara todos los valores y tuviera la mediana correspondiente. Los fragmentos de código y las salidas capturadas corresponden a Programa1.c.

Desarrollo y resultados de inserción

Inserción mantiene una parte inicial ordenada. En cada iteración guarda el siguiente valor en key, desplaza a la derecha los elementos mayores y coloca key en el espacio que le corresponde.

```
void insercion(int arreglo[], int m){
    int i, j, key;
    for (j = 1; j < m; j++){
        key = arreglo[j];
        i = j - 1;
        while ((i > -1) && (arreglo[i] > key)){
            arreglo[i + 1] = arreglo[i];
            i = i - 1;
        }
        arreglo[i + 1] = key;
    }
}
```

Figura 2. Función insercion. La condición $i > -1$ impide recorrer posiciones anteriores al inicio del arreglo.

La versión por pasos presenta dos columnas, siguiendo la idea de acomodar cartas: a la izquierda están los valores ya insertados y a la derecha los pendientes. Los valores iguales permanecen en el arreglo.

Entrada									
	-52		61		-52		-54		97 -40
Arreglo Ordenado					Datos por Ordenar				
	-52					61		-52	-54 97 -40
	-52		61			-52		-54	97 -40
	-52		-52		61		-54		97 -40
	-54		-52		-52		61		97 -40
	-54		-52		-52		61		97
	-54		-52		-52		-40		61 97
La mediana en el arreglo de numero es: -46.00									
Arreglo ordenado									
	-54		-52		-52		-40		61 97

Figura 3. Salida de inserción con seis valores aleatorios. El -54 se coloca al inicio y ambos -52 se conservan. La mediana es $(-52 + -40) / 2 = -46$. Se recortó únicamente espacio vacío entre las columnas para mejorar la lectura.

Desarrollo y resultados de burbuja

Burbuja compara elementos vecinos y los intercambia cuando están invertidos. Al terminar una pasada, el mayor valor pendiente queda al final; por eso el límite interior disminuye con j . La bandera b permite terminar cuando una pasada completa no produce intercambios.

```
void burbuja(int arreglo[], int m){
    int aux,b;
    for (int j = 0; j < m-1; j++)
    {
        b=0;
        for (int i = 0; i < m-1-j; i++)
        {
            if(arreglo[i]>arreglo[i+1]){
                aux=arreglo[i];
                arreglo[i]=arreglo[i+1];
                arreglo[i+1]=aux;
                b=1;
            }
        }
        if(b==0)
            return;
    }
}
```

Figura 4. Función burbuja con límite $m - 1 - j$ y terminación anticipada mediante b .

Entrada						
	5	4	3	2	1	
Inicial:	5	4	3	2	1	
Pasada 1:	4	3	2	1	5	
Pasada 2:	3	2	1	4	5	
Pasada 3:	2	1	3	4	5	
Pasada 4:	1	2	3	4	5	
La mediana en el arreglo de numero es: 3.00						
Arreglo ordenado						
	1	2	3	4	5	

Figura 5. Salida de burbuja con la entrada invertida 5, 4, 3, 2, 1. Se requieren cuatro pasadas con intercambios; primero queda fijo el 5 y después el 4, el 3 y el 2. La mediana final es 3.

Desarrollo y resultados de selección

Selección busca el menor elemento del tramo pendiente. La variable min guarda su valor y k su posición. Después de recorrer ese tramo, lo intercambia con el elemento de la posición j. Así, cada pasada deja una posición inicial definitivamente ordenada.

```
void seleccion(int arreglo[], int m){
    int min,k,j;
    for (j = 0; j < m-1; j++)
    {
        min=arreglo[j];
        k=j;
        for (int i = j+1; i < m; i++)
        {
            if(arreglo[i]<min)
            {
                min = arreglo[i];
                k=i;
            }
        }
        if (k!=j)
        {
            arreglo[k]=arreglo[j];
            arreglo[j]=min;
        }
    }
}
```

Figura 6. Función seleccion. El recorrido interior incluye el último elemento y el intercambio se realiza al terminar la búsqueda del mínimo.

Entrada		-52		61		-52		-54		97		-40	
Inicial:		-52		61		-52		-54		97		-40	
Pasada 1:		-54		61		-52		-52		97		-40	
Pasada 2:		-54		-52		61		-52		97		-40	
Pasada 3:		-54		-52		-52		61		97		-40	
Pasada 4:		-54		-52		-52		-40		97		61	
Pasada 5:		-54		-52		-52		-40		61		97	
La mediana en el arreglo de numero es: -46.00													
Arreglo ordenado		-54		-52		-52		-40		61		97	

Figura 7. Salida de selección con la misma entrada aleatoria usada en inserción. La primera pasada coloca -54 al inicio. Las pasadas sin intercambio también se imprimen; el resultado conserva los dos valores -52.

Desarrollo del ordenamiento por mezcla

Mezcla divide el arreglo en dos mitades y ordena cada una mediante llamadas recursivas. Cuando el tramo tiene un solo elemento, la condición $p < r$ es falsa y la llamada termina. Después, merge combina las dos mitades ordenadas.

```
void mezcla(int array[], int p, int r){
    if (p < r){
        int q = p + (r - p) / 2;

        mezcla(array, p, q);
        mezcla(array, q + 1, r);

        merge(array, p, q, r);
    }
}
```

Figura 8. Función mezcla. Los índices p y r son inclusivos; para el arreglo completo se llama mezcla(arreglo, 0, m - 1).

merge reserva los arreglos auxiliares L y R con malloc y copia en ellos las mitades. Compara los primeros valores pendientes y escribe el menor en el arreglo original. Si una mitad se agota, copia los restantes de la otra. Al terminar libera ambos auxiliares con free.

```
// Fusión de datos respetando el valor minimos entre dos arreglos
for (k = p; k <= r; k++)
{
    if (i == n_1)
    {
        array[k] = R[j];
        j = j + 1;
    }
    else if(j == n_2)
    {
        array[k] = L[i];
        i = i + 1;
    }
    else
    {
        if (L[i] <= R[j])
        {
            array[k] = L[i];
            i = i + 1;
        }
        else
        {
            array[k] = R[j];
            j = j + 1;
        }
    }
}
```

Figura 9. Fragmento de merge que une L y R. i y j recorren los auxiliares, mientras k indica la posición de escritura en el arreglo original.

Resultados del ordenamiento por mezcla

La versión mezclaPasos muestra las mitades L y R que se van a unir, seguidas del tramo resultante. Las primeras uniones abarcan tramos pequeños; la última reúne el arreglo completo. Cada mitad ya está ordenada cuando se combina con la otra.

```
Entrada
|  -52 |   61 |  -52 |  -54 |   97 |  -40 |

L: |  -52 |
R: |   61 |
Resultado: |  -52 |   61 |

L: |  -52 |   61 |
R: |  -52 |
Resultado: |  -52 |  -52 |   61 |

L: |  -54 |
R: |   97 |
Resultado: |  -54 |   97 |

L: |  -54 |   97 |
R: |  -40 |
Resultado: |  -54 |  -40 |   97 |

L: |  -52 |  -52 |   61 |
R: |  -54 |  -40 |   97 |
Resultado: |  -54 |  -52 |  -52 |  -40 |   61 |   97 |

La mediana en el arreglo de numero es: -46.00

Arreglo ordenado
|  -54 |  -52 |  -52 |  -40 |   61 |   97 |
```

Figura 10. Salida de mezcla con seis valores aleatorios. La última unión combina $L = [-52, -52, 61]$ y $R = [-54, -40, 97]$. El resultado coincide con los demás métodos y su mediana es -46 .

Las dos apariciones de -52 se conservan. La comparación $L[i] \leq R[j]$ toma primero el elemento de la izquierda cuando ambos tienen el mismo valor.

Resultados con entradas especiales

Arreglo previamente ordenado

```
Entrada
| 1 | 2 | 3 | 4 | 5 | 6 |

Inicial: | 1 | 2 | 3 | 4 | 5 | 6 |

La mediana en el arreglo de numero es: 3.50

Arreglo ordenado
| 1 | 2 | 3 | 4 | 5 | 6 |
```

Figura 11. Burbuja recibe los valores del 1 al 6 y los conserva. La mediana es $(3 + 4) / 2 = 3.50$.

La primera pasada no realiza intercambios, de modo que b permanece en cero y la función termina. En burbujaPasos esa comprobación está antes de imprimir la pasada; por eso la salida muestra el estado inicial y el resultado, sin una fila adicional de cambios.

Arreglo de un elemento

```
Entrada
| 1 |

La mediana en el arreglo de numero es: 1.00

Arreglo ordenado
| 1 |
```

Figura 12. Mezcla recibe un único valor, 1. No aparecen uniones L y R y la mediana también es 1.

El arreglo ya está ordenado. Al iniciar con $p = r = 0$, la función no divide ni llama a merge. Este caso comprueba que el algoritmo respeta su condición de terminación.

Verificación de las funciones

Se comprobaron 140 combinaciones entre los cuatro métodos, siete tamaños y cinco tipos de datos: ordenados, invertidos, valores entre -100 y 100, todos iguales y combinaciones de -100, 0 y 100. Los tamaños fueron 1, 2, 6, 10, 11, 100 y 10 000. En todos los casos se conservó el contenido, se obtuvo orden ascendente y la mediana coincidió con la calculada a partir del resultado esperado.

Resultados con más de diez elementos

Cuando la cantidad supera diez, el programa utiliza las funciones sin pasos. El tiempo se calcula con `clock()` alrededor del ordenamiento y se convierte a milisegundos enteros. La generación de datos, la copia del arreglo original y la impresión del resultado se realizan fuera de ese intervalo.

```
Algoritmo de ordenamiento ejecutado con exito

Tiempo de ordenamiento: 0 ms

La mediana en el arreglo de numero es: -23.50
```

Figura 13. Fragmento de la salida de mezcla para 20 valores aleatorios. El resultado tuvo mediana -23.50 y la duración mostrada fue 0 ms.

Un valor de 0 ms no significa ausencia de trabajo: puede deberse a la resolución de `clock()` y a la conversión del resultado a entero. Por ello, este caso sirve para comprobar el cambio a la versión sin pasos, pero no para establecer diferencias de velocidad entre algoritmos.

Comparación con 10 000 elementos

La tabla registra una ejecución por método y tipo de entrada, en Windows y con GCC sin optimizaciones. Para los aleatorios se fijó la misma semilla de prueba y se partió de los mismos valores. Las cifras describen estas ejecuciones y pueden variar según el equipo y la carga del sistema.

Método	Ordenado ms	Invertido ms	Aleatorio ms
Inserción	0	51	24
Burbuja	0	94	93
Selección	26	35	26
Mezcla	1	2	3

Inserción y burbuja aprovechan una entrada ya ordenada: inserción no desplaza elementos y burbuja termina tras su primera revisión. Selección continúa buscando el mínimo en cada tramo. Mezcla sigue dividiendo y uniendo las mitades; en las entradas invertida y aleatoria de esta prueba presentó los menores tiempos registrados.

Estos comportamientos corresponden a los recorridos del código: inserción y burbuja tienen un mejor caso lineal y un peor caso cuadrático; selección realiza un número cuadrático de comparaciones; mezcla divide y combina en un tiempo de orden $n \log n$, a cambio de memoria auxiliar proporcional al tamaño del arreglo.

Referencia de la actividad: Cecilia Albortante Morato, Práctica 2 — Ordenamiento por inserción, burbuja, selección y mezcla, presentación proporcionada para la asignatura, diapositiva 2. Código utilizado: Programa1.c de la práctica 2.